

Universidad del Valle
Departamento de Ingeniería de Sistemas Informáticos

Chameleons

Implementación de Machine Learning en Unity

Materia

Game Development

Docente

Erick Salomon Cespedes Gamarra

Integrantes

Saúl Álvarez Alanes

Estefanía Laredo Hinojosa

Motor

Unity 6000.5.5f1 · C#

Repositorio

github.com/EstefaniaLaredo/Chameleons

1. Introducción

1.1 Objetivo de la práctica

El objetivo de esta práctica fue implementar un sistema básico de machine learning dentro de un videojuego corto, en el que un conjunto de células intenta sobrevivir a los ataques del jugador adaptándose a su entorno. El resultado es **Chameleons**, un juego desarrollado en Unity en el que el jugador elimina células haciendo clic sobre ellas durante rondas de diez segundos, mientras las células aprenden, ronda tras ronda, qué colores y qué tamaños les permiten pasar desapercibidas.

1.2 Investigación: el machine learning aplicado a videojuegos

El machine learning es la rama de la inteligencia artificial que permite a un sistema mejorar su comportamiento a partir de la experiencia, en lugar de seguir reglas escritas por un programador. Se suele dividir en tres grandes familias:

- **Aprendizaje supervisado:** el sistema aprende a partir de ejemplos ya etiquetados con la respuesta correcta. Se usa, por ejemplo, para clasificar imágenes.
- **Aprendizaje no supervisado:** el sistema busca patrones en datos sin etiquetar, como cuando se agrupan jugadores por su estilo de juego.
- **Aprendizaje por refuerzo:** un agente toma decisiones, recibe una recompensa o un castigo según el resultado, y ajusta su comportamiento para maximizar la recompensa acumulada. No se le dice qué hacer; descubre qué funciona probando.

En videojuegos, el aprendizaje por refuerzo es el enfoque dominante porque el juego ya provee de forma natural los tres elementos que necesita: un **estado** (la situación actual), un conjunto de **acciones** posibles y una **señal de recompensa** (sobrevivir, puntuar, ganar). Casos conocidos son los agentes de DeepMind en Atari y AlphaGo, o sistemas comerciales de dificultad adaptativa que ajustan el reto según el desempeño del jugador.

Nuestro problema encaja exactamente en ese molde. Cada célula es un agente que elige un color y un tamaño (sus acciones), y al final de la ronda descubre si esa elección la mantuvo con vida (su recompensa). No existe una respuesta correcta escrita en ningún lado: la respuesta depende del fondo de la escena y de la habilidad del jugador, y solo puede descubrirse jugando.

1.3 Por qué no se usó Unity ML-Agents

Unity ofrece el paquete ML-Agents, que permite entrenar redes neuronales mediante aprendizaje por refuerzo profundo. Se descartó deliberadamente por tres razones:

- Requiere una instalación paralela de Python y un ciclo de entrenamiento previo de horas o días, cuyo resultado es un modelo ya entrenado. El aprendizaje no ocurriría durante la partida, que es justamente lo que el enunciado pide demostrar.
- El enunciado solicita explícitamente un *algoritmo simple de aprendizaje* en el que las células registren si lograron sobrevivir y ajusten su color en consecuencia. Eso describe un método tabular, no una red neuronal.

- Un modelo entrenado por ML-Agents es una caja negra: no se puede explicar por qué eligió un color. Nuestra implementación permite abrir el Inspector durante la partida y ver los puntajes cambiando en tiempo real.

Se implementó por tanto un algoritmo de aprendizaje por refuerzo tabular propio, escrito íntegramente en C#, del tipo conocido como **epsilon-greedy**.

2. Conceptualización del juego

2.1 Mecánicas

La partida se divide en **rondas de diez segundos**. Al comenzar cada ronda aparece una generación de doce células distribuidas al azar por la pantalla. El jugador dispone de esos diez segundos para eliminar las que alcance a ver. Cuando el tiempo se agota, las células que siguen vivas reportan su éxito, todas desaparecen, y nace una nueva generación que hereda lo aprendido por la anterior.

El fondo de la escena es de un color fijo. Ese detalle es el que da sentido a todo el sistema: las células cuyo color se parece al fondo son más difíciles de ver, sobreviven más y, en consecuencia, su color sube de puntaje. La partida no tiene final: es una sucesión indefinida de generaciones, y el interés está en observar cómo la población cambia.

2.2 Interacción del jugador

La única acción del jugador es el **clic izquierdo sobre una célula**. La detección se resuelve con el método `OnMouseDown()` de Unity, que se dispara cuando el jugador presiona el botón del ratón sobre un objeto que tiene un collider. Cada célula lleva un collider ajustado a su forma, por lo que el clic solo cuenta si realmente acierta sobre ella.

Un contador en la parte superior de la pantalla registra cuántas células ha eliminado el jugador en total, junto al número de ronda actual y al tiempo restante.

2.3 Parámetros del juego

Parámetro	Valor	Descripción
Duración de la ronda	10 s	Tiempo que tiene el jugador para eliminar células
Células por ronda	12	Generación que nace al inicio de cada ronda
Máximo en pantalla	20	Límite duro de optimización
Tamaño mínimo	0.25	Célula más pequeña posible
Tamaño máximo	1.00	Célula más grande posible
Genes de color	12	Colores distintos entre los que puede elegir
Genes de tamaño	5	Tamaños distintos entre los que puede elegir
Velocidad de deriva	0.4	Desplazamiento lento para que no sean estáticas

3. Arquitectura del proyecto

3.1 Estructura de carpetas

```
Assets/  
  00_Scenes/      Game.unity, la escena principal  
  01_Scripts/     Los cinco scripts del proyecto  
  02_Prefabs/     Cell.prefab, la plantilla de la célula  
  03_Sprites/     Cell_Sprite.png y Square.png  
  Settings/       Configuración del render pipeline (URP 2D)  
Packages/         Dependencias del proyecto  
ProjectSettings/ Configuración de Unity
```

3.2 Los cinco scripts

Script	Responsabilidad	Autor
GameManager.cs	Control de rondas, puntaje e interfaz	Saúl
CellSpawner.cs	Creación y destrucción de las generaciones	Saúl
Cell.cs	Comportamiento individual de cada célula	Saúl
CellBrain.cs	Algoritmo de aprendizaje compartido	Estefanía
LearningStats.cs	Panel de estadísticas del aprendizaje	Estefanía

La separación fundamental es entre **Cell** y **CellBrain**. Cada célula es un objeto físico en la escena que nace y muere en cada ronda; el cerebro es un objeto único que persiste durante toda la partida y guarda la memoria de la población. Si el conocimiento viviera dentro de cada célula, moriría con ella y no habría aprendizaje posible entre generaciones.

3.3 La escena

La escena `Game.unity` contiene los siguientes objetos:

- **Main Camera**: cámara ortográfica con tamaño 5.
- **Background**: un cuadrado blanco escalado para cubrir la pantalla y teñido del color de fondo, con orden de dibujo -10 para quedar detrás de todo.
- **GameManager**: objeto vacío que lleva los componentes `GameManager` y `CellSpawner`.
- **CellBrain**: objeto vacío que lleva los componentes `CellBrain` y `LearningStats`.
- **Canvas**: contiene el panel superior con los contadores de ronda, tiempo y células eliminadas, y el panel inferior izquierdo con las estadísticas del aprendizaje.

4. El sistema de machine learning

4.1 El problema que resuelve

La población de células debe descubrir, sin que nadie se lo diga, qué combinación de color y tamaño maximiza su probabilidad de sobrevivir. El sistema no conoce el color del fondo, no sabe que los objetos pequeños son más difíciles de clicar, y no tiene acceso a ninguna información sobre el jugador. Todo lo que recibe es, al final de cada ronda, un dato por célula: **sobrevivió o fue eliminada**.

4.2 Representación del conocimiento

El conocimiento se guarda en dos arreglos de números reales dentro de CellBrain:

```
public float[] colorScore;    // un puntaje por cada uno de los 12 colores
public float[] sizeScore;    // un puntaje por cada uno de los 5 tamaños
```

Ambos arreglos se crean vacíos en `Awake()`, es decir, con todos sus valores en cero. Eso representa el estado inicial de ignorancia: al empezar la partida, ningún color vale más que otro.

El espacio de colores se discretiza en doce **genes**. El gen número *i* corresponde a un tono (hue) interpolado linealmente entre los límites mínimo y máximo definidos:

```
public Color GetColor(int index)
{
    float hue = Mathf.Lerp(minHue, maxHue, (float)index / (colorGenes - 1));
    return Color.HSVToRGB(hue, saturation, brightness);
}
```

El tamaño sigue exactamente la misma idea con cinco genes entre `minSize` y `maxSize`. Esta discretización es lo que cumple el requisito del enunciado de establecer límites mínimos y máximos tanto para el tamaño como para los valores de color.

4.3 El algoritmo: epsilon-greedy

Cuando una célula nace, le pide al cerebro qué color y qué tamaño usar. La decisión resuelve el problema clásico del aprendizaje por refuerzo conocido como el **dilema entre explorar y explotar**: si el sistema repite siempre lo que ya conoce, nunca descubrirá algo mejor; si prueba siempre al azar, nunca aprovechará lo aprendido.

```
public int ChooseColor()
{
    if (Random.Range(0f, 1f) < explorationRate)
    {
        return Random.Range(0, colorGenes);    // EXPLORAR
    }

    return BestIndex(colorScore);                // EXPLOTAR
}
```

Con probabilidad `explorationRate` la célula elige un gen al azar. El resto de las veces elige el gen con el puntaje más alto acumulado hasta ese momento. Este método se llama

epsilon-greedy, donde epsilon es precisamente esa probabilidad de explorar.

4.4 La función de recompensa

Cada célula reporta su resultado exactamente una vez por ronda: al morir, o al terminar la ronda si sigue viva.

```
public void Learn(int colorIndex, int sizeIndex, bool alive)
{
    roundBorn++;

    if (alive)
    {
        colorScore[colorIndex] += reward;    // +1 al color que sobrevivio
        sizeScore[sizeIndex]   += reward;    // +1 al tamaño que sobrevivio
        roundSurvived++;
    }
    else
    {
        colorScore[colorIndex] -= penalty;    // -1 al color que fue visto
        sizeScore[sizeIndex]   -= penalty;
    }
}
```

El puntaje de un gen es, por tanto, el número de veces que sobrevivió menos el número de veces que fue eliminado. Un gen con puntaje alto es un gen que históricamente ha funcionado. Este es el **sistema de evaluación** que pide el enunciado: la población aprende tanto de sus intentos exitosos como de los fallidos.

4.5 El decaimiento de la exploración

La tasa de exploración no es constante. Comienza en 0.9 (noventa por ciento de decisiones al azar) y disminuye un 0.12 en cada ronda, hasta un mínimo de 0.1:

```
public void NextRound()
{
    lastBorn = roundBorn;
    lastSurvived = roundSurvived;
    roundBorn = 0;
    roundSurvived = 0;

    explorationRate = explorationRate - explorationDecay;

    if (explorationRate < explorationMin)
    {
        explorationRate = explorationMin;
    }
}
```

El razonamiento es intuitivo: al principio la población no sabe nada, así que le conviene probar mucho. A medida que acumula evidencia, le conviene confiar cada vez más en lo aprendido. El mínimo del diez por ciento evita que el sistema se cierre por completo, para que pueda seguir adaptándose si el jugador cambia de estrategia.

4.6 El ciclo completo de una ronda

Momento	Qué ocurre	Método responsable
Inicio	Nacen 12 células en posiciones aleatorias	CellSpawner.SpawnRound()
Nacimiento	Cada célula pide su color y tamaño al cerebro	CellBrain.ChooseColor() / ChooseSize()
Durante	El jugador hace clic; la célula que muere reporta fracaso	Cell.OnMouseDown()
Fin del tiempo	Las vivas reportan éxito y se destruyen	CellSpawner.EndRound()
Cierre	Se guardan los resultados y baja la exploración	CellBrain.NextRound()
Reinicio	Nace la siguiente generación	GameManager.EndRound()

4.7 Por qué esto es aprendizaje y no azar

Un observador escéptico podría argumentar que las células simplemente cambian de color al azar. Tres hechos observables lo desmienten:

- **La convergencia es reproducible.** Partida tras partida, la población termina eligiendo tonos cercanos al color del fondo y los tamaños más pequeños. El azar no produce resultados consistentes.
- **La tasa de supervivencia sube.** El panel muestra cuántas células sobrevivieron la ronda anterior; ese número crece con las generaciones aunque el jugador no cambie.
- **El comportamiento responde al entorno.** Si se cambia el color del fondo en el Inspector, la población converge hacia un color distinto. El sistema se adapta al entorno, que es la definición misma de aprendizaje.

5. Lógica de adaptación de las células

5.1 Cómo aplica la célula lo que el cerebro decide

Toda la adaptación ocurre en el método `Start()` de `Cell.cs`, en el instante en que la célula nace:

```
void Start()
{
    sr = GetComponent<SpriteRenderer>();

    colorIndex = CellBrain.instance.ChooseColor();
    sizeIndex = CellBrain.instance.ChooseSize();

    sr.color = CellBrain.instance.GetColor(colorIndex);

    float size = CellBrain.instance.GetSize(sizeIndex);
    transform.localScale = new Vector3(size, size, 1f);

    direction = new Vector3(Random.Range(-1f, 1f),
                            Random.Range(-1f, 1f), 0f).normalized;
}
```

La célula guarda los índices elegidos en `colorIndex` y `sizeIndex` porque los necesitará al morir: son la única forma de decirle al cerebro *qué* gen falló o funcionó.

5.2 El sprite en escala de grises

Un detalle de implementación que hace posible la adaptación del color: el sprite de la célula está dibujado en **escala de grises**, no en color. La propiedad `sr.color` de Unity multiplica el color del sprite por el color asignado, de modo que un sprite blanco puede teñirse de cualquier tono, mientras que uno ya coloreado distorsionaría el resultado.

5.3 El reporte del resultado

La célula tiene una bandera `reported` que garantiza que informe exactamente una vez, evitando que una misma célula sume y reste puntaje en la misma ronda:

```
void Report(bool alive)
{
    if (reported)
    {
        return;
    }

    reported = true;
    CellBrain.instance.Learn(colorIndex, sizeIndex, alive);
}
```

5.4 Resultado observado

Durante las pruebas, la evolución típica de una partida es la siguiente:

Rondas	Comportamiento observado
1 a 3	Células de todos los colores y tamaños. Exploración del 90 al 66 por ciento. El jugador elimina casi todas.
4 a 7	Empiezan a dominar los tonos verdosos y los tamaños medianos y pequeños. La supervivencia sube.
8 en adelante	La mayoría de las células son de un tono muy cercano al del fondo y del tamaño mínimo. Cuesta encontrarlas

6. Pruebas y optimización

6.1 Pruebas realizadas

- Se verificó que las células aparecen dentro de los límites de la pantalla y que rebotan al llegar al borde en lugar de salirse.
- Se comprobó que el clic elimina la célula y que el contador de eliminadas sube en uno.
- Se comprobó que al terminar la ronda las células supervivientes desaparecen y nace una nueva generación, y que el contador de rondas avanza.
- Se observó el arreglo `colorScore` desde el Inspector durante la partida para confirmar que los puntajes suben y bajan correctamente.
- Se verificó que la tasa de exploración desciende ronda a ronda y se detiene en el mínimo.

6.2 Ajuste de parámetros

El equilibrio del juego se logró ajustando tres valores. Con una tasa de decaimiento demasiado baja el aprendizaje era imperceptible en una demostración corta; con una demasiado alta la población se cerraba en el primer color que le funcionara por casualidad. El valor de 0.12 por ronda permite ver la convergencia en menos de diez rondas sin perder el carácter exploratorio.

De forma similar, doce células por ronda resultó ser el número que hace el juego exigente sin volverlo imposible, y un tamaño mínimo de 0.25 mantiene las células visibles pero difíciles de acertar.

6.3 Optimizaciones implementadas

- **Límite de células en pantalla.** El spawner nunca instancia más de `maxCells` células, sin importar el valor de `cellsPerRound`. Evita que un valor mal configurado degrade el rendimiento.
- **Destrucción de la generación completa.** Al terminar la ronda todas las células se destruyen, de modo que el número de objetos activos nunca crece con el tiempo.
- **Algoritmo de coste constante.** Elegir un gen y aprender de un resultado son operaciones sobre arreglos pequeños de tamaño fijo. El coste del aprendizaje no aumenta con el número de rondas jugadas, a diferencia de un sistema que guardara el historial completo.
- **Referencia única al cerebro.** El cerebro es un singleton accesible mediante `CellBrain.instance`, por lo que las células no tienen que buscarlo en la escena cada vez que nacen.

7. Control de versiones y gestión del proyecto

7.1 Git

El proyecto se gestionó con Git siguiendo la estructura de ramas indicada por el docente en su guía:

Rama	Función
main	Únicamente versiones finales del proyecto
dev	Rama principal de trabajo; de ella nacen las demás
feature_JuegoBase	Juego base: rondas, spawner, interacción e interfaz
feature_MachineLearning	Sistema de aprendizaje y panel de estadísticas

Cada integrante trabajó en su propia rama creada a partir de dev, con commits descriptivos de menos de diez palabras, y unió su trabajo a dev al terminar. Se incluyó un archivo `.gitignore` para Unity, de modo que las carpetas generadas automáticamente (Library, Temp, Logs) y los archivos de solución de Visual Studio no se suban al repositorio.

7.2 Trello

El avance se controló en un tablero de Trello con cuatro listas siguiendo el diagrama de flujo del docente: **To Do**, **Doing**, **To Test** y **Done**. Cada tarea corresponde a una rama del repositorio; la tarjeta pasa a To Test cuando la rama se sube, y a Done cuando se une a dev y se verifica su funcionamiento. Si al probarla se encuentran defectos, vuelve a To Do con las observaciones anotadas.

7.3 División del trabajo

Integrante	Responsabilidad
Saúl Álvarez Alanes	Juego base: control de rondas de diez segundos, generación de células, interacción por clic, con
Estefanía Laredo Hinojosa	Sistema de machine learning: algoritmo epsilon-greedy, tablas de puntaje por color y tamaño, fun

8. Conclusiones

El proyecto demuestra que no hace falta una red neuronal ni una biblioteca externa para que un sistema aprenda dentro de un videojuego. Un algoritmo de aprendizaje por refuerzo tabular de poco más de cien líneas de C# es suficiente para que una población de agentes descubra, por sí sola y durante la partida, una estrategia que nadie programó: camuflarse con el fondo y reducir su tamaño.

El valor pedagógico está precisamente en esa simplicidad. Cada decisión del sistema puede abrirse, leerse y justificarse línea por línea, y el aprendizaje puede observarse ocurriendo en tiempo real en el panel de estadísticas, en lugar de quedar oculto dentro de los pesos de un modelo.

La principal limitación del enfoque es que el aprendizaje es de la población y no individual: una célula concreta no mejora durante su vida, sino que hereda el conocimiento acumulado por las anteriores. Una extensión natural sería incorporar el estado del entorno a la decisión —por ejemplo, la zona de la pantalla en la que nace la célula— convirtiendo las tablas actuales en una tabla de Q-learning con estados.

Anexo. Parámetros configurables desde el Inspector

Todos los valores del algoritmo son públicos y pueden modificarse sin tocar el código, lo que permitió ajustar el equilibrio del juego durante las pruebas.

Variable	Valor	Qué controla
minHue / maxHue	0 / 1	Límites del rango de color disponible
saturation	0.6	Saturación fija de todos los colores
brightness	0.8	Brillo fijo de todos los colores
colorGenes	12	Cuántos colores distintos puede elegir
minSize / maxSize	0.25 / 1	Límites del rango de tamaño
sizeGenes	5	Cuántos tamaños distintos puede elegir
explorationRate	0.9	Probabilidad inicial de probar al azar
explorationMin	0.1	Exploración mínima que nunca se abandona
explorationDecay	0.12	Cuánto baja la exploración por ronda
reward	1	Puntos que gana un gen al sobrevivir
penalty	1	Puntos que pierde un gen al ser eliminado
roundTime	10	Duración de la ronda en segundos
cellsPerRound	12	Células que nacen por ronda
maxCells	20	Límite duro de células en pantalla